

DEPLOYMIND

AI-Assisted Incident Response with Persistent Operational Memory

Project Documentation

1. Introduction

DeployMind is an AI-assisted incident-response system designed to help DevOps engineers investigate production incidents by combining current incident evidence with reusable operational experience. The system is intentionally designed as an orchestration workflow rather than as a standalone chatbot. Its central loop is: Current Incident → Hindsight Recall → AI Reasoning → Human Approval → Resolution → Hindsight Retain.

The architecture separates responsibilities between the React interface, the Java Spring Boot backend, Hindsight memory, the Groq language model, and the human engineer. Current incident information remains the primary source of truth, while historical memory is treated as supporting evidence. The AI produces an investigation and recommendation, but it does not execute production changes.

2. Objectives

- Provide a structured workspace for investigating production incidents.
- Use historical operational experience to improve incident investigation.
- Combine current incident evidence and recalled memory before generating a recommendation.
- Keep the final production decision with a human engineer.
- Retain verified resolution outcomes so they can support future incidents.
- Provide a deterministic evidence-first fallback when the LLM or historical memory is unavailable.

3. Technology Stack

The technology stack used in the supplied DeployMind project is listed separately for clarity.

Layer	Technology	Purpose
Frontend	React + Vite	Incident workspace, incident creation, memory evidence, investigation and approval interface.
Backend	Java 17 + Spring Boot 3.2.5	REST API, validation, orchestration and incident lifecycle control.
Memory	Hindsight Cloud REST API	Stores and recalls operational incident experience.
LLM	Groq OpenAI-compatible Chat Completions	Provides structured incident reasoning and recommendations.
Fallback	Local demo memory and deterministic investigation plans	Keeps the demonstration usable when external services are not configured.
Configuration	Environment variables and application.properties	Configures API keys, endpoints, model and service settings.

4. System Architecture

DeployMind follows a clear request path. The React + Vite interface sends an incident request to the Spring Boot REST controller. The backend loads the current incident, creates a contextual memory query, retrieves relevant historical experience from Hindsight, and passes the current facts and retrieved evidence to the reasoning service. The result is returned as structured data to the frontend. After human review and approval, the resolution outcome is retained as future operational memory.

Architecture diagram

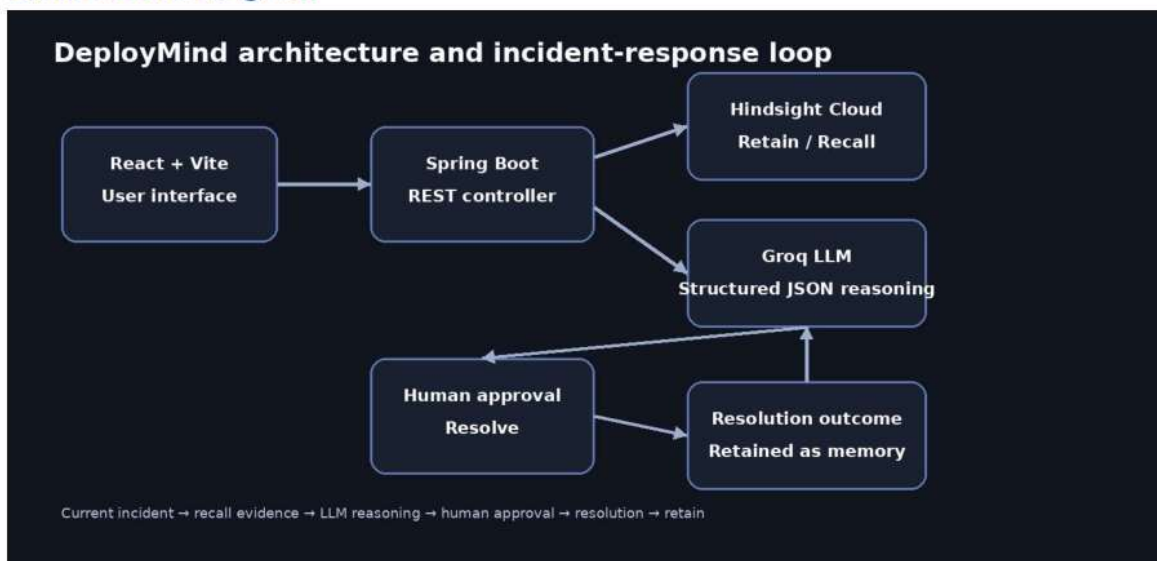


Figure 1. DeployMind architecture and incident-response loop.

4.1 Architecture Components

Component	Responsibility
React + Vite	Provides the incident workspace, new-incident form, memory evidence page, recommendation display and approval action.
Spring Boot REST Controller	Acts as the application boundary and controls incident lookup, memory recall, analysis and resolution.
Hindsight Cloud	Stores and recalls operational experience through Retain and Recall operations.
Groq LLM	Converts current incident facts and retrieved evidence into structured incident analysis.
AgentService	Classifies incidents, filters relevant memory, calls the LLM and provides the deterministic fallback plan.

IncidentService	Maintains the incident collection and manages incident creation, lookup and resolution.
Human Approval	Provides the authority boundary before a recommendation is accepted and the outcome is retained.

5. Incident Data Model

An incident contains the context required for meaningful investigation. The supplied implementation represents an incident with an identifier, service, title, severity, status, deployment, error, symptoms, recent changes, creation time, root cause and resolution. This structure allows the memory query and reasoning process to use more than an isolated error message.

Field	Description
id	Unique incident identifier.
service	Affected application or service.
title	Short incident title.
severity	Incident severity.
status	Current incident lifecycle state.
deployment	Deployment or release associated with the incident.
error	Observed error or log message.
symptoms	Observed operational symptoms.
recentChanges	Recent configuration or deployment changes.
rootCause	Verified root cause recorded during resolution.
resolution	Recorded resolution outcome.

6. Incident Investigation Workflow

1. The engineer selects an open incident from the React workspace.
2. The frontend sends the incident ID and memory-use flag to the `/api/analyze` endpoint.

3. Spring Boot loads the current incident and builds a contextual recall query from the service, error, symptoms, recent changes, and the concepts of root cause and resolution.
4. Hindsight returns historical operational experience when memory is enabled.
5. AgentService filters relevant memories and classifies the incident category.
6. The configured Groq LLM receives structured current facts and relevant historical evidence and returns structured JSON reasoning.
7. If the LLM is unavailable, the deterministic evidence-first investigation plan is used.
8. The recommendation, rationale, investigation steps, evidence to check, and memory information are displayed in the React interface.
9. The engineer reviews the recommendation and decides whether to approve the resolution.
10. The verified outcome is retained as operational memory for future incidents.

7. Hindsight Memory Integration

Hindsight has a focused responsibility in DeployMind: it stores and recalls operational experience. The backend supports Hindsight Cloud through a REST API. When an API key is configured, the application uses the Hindsight backend. When the key is not configured, the supplied implementation provides a local demo memory fallback so the workflow can still be demonstrated.

For recall, the backend constructs a query using the current incident service, error, symptoms, recent changes, and the concepts of root cause and resolution. The implementation requests up to five recalled results for the investigation. During resolution, the backend stores the incident outcome with incident metadata so that the result can become future evidence.

The architecture intentionally treats memory as evidence rather than authority. A historical solution may be relevant to a new incident without being universally correct, so current incident facts remain primary.

8. LLM Integration and Human-in-the-Loop Control

The current implementation uses Groq as the LLM provider through an OpenAI-compatible Chat Completions endpoint. The backend can be configured with `GROQ_API_KEY` and `LLM_MODEL`. Structured output is used so that the frontend receives predictable fields instead of parsing free-form text.

The system prompt establishes a strict authority boundary: the model does not execute production actions; a human engineer makes the final decision; current incident facts are primary; Hindsight memories are supporting evidence; and the model must not invent infrastructure state, metrics, logs, capacity, or completed actions.

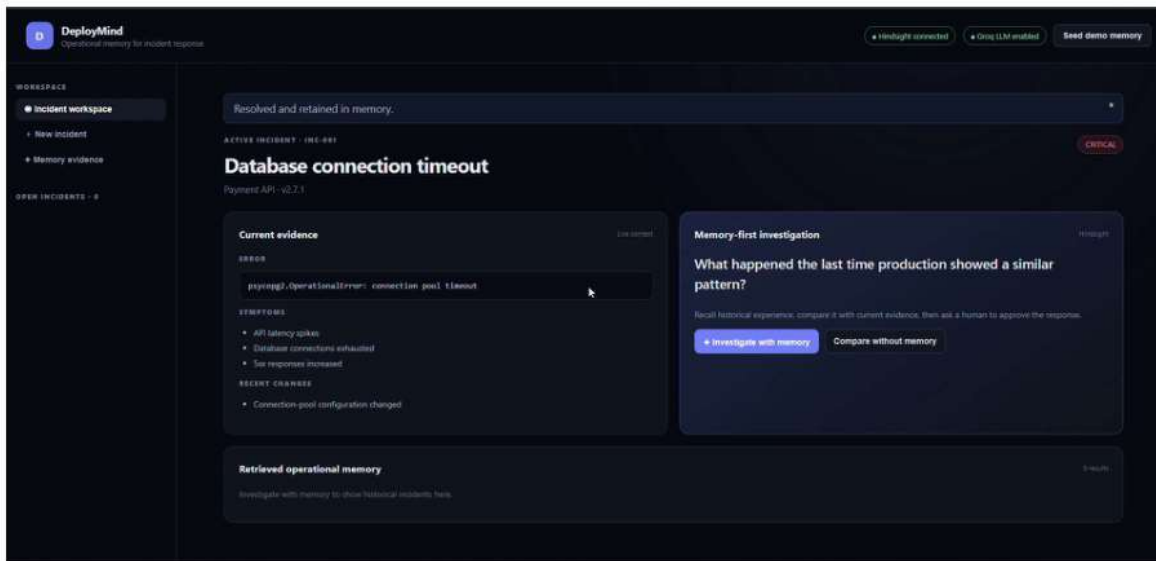


Figure 2. DeployMind incident workspace showing current evidence and the memory-first investigation interface.

9. Key Backend API Endpoints

Endpoint	Method	Purpose
/api/health	GET	Reports backend health, memory backend, LLM status, and human-approval requirement.

DeployMind — Project Documentation

/api/incidents	GET	Returns available incidents.
/api/incidents	POST	Creates a new incident.
/api/metrics	GET	Returns incident counts and configuration-related metrics.
/api/seed-demo	POST	Seeds resolved incident experiences into the configured memory backend.
/api/judge-challenge	POST	Creates the supplied challenge incident for demonstration.
/api/analyze	POST	Runs incident analysis with or without historical memory.
/api/resolve	POST	Approves and resolves an incident and retains the verified outcome.

10. Frontend Features

- Incident workspace for selecting and inspecting open incidents.
- New incident form for creating realistic production problems.
- Memory-first investigation action and comparison without memory.
- Recommendation panel showing classification, confidence, rationale, investigation steps, and evidence to check.
- Before-versus-after memory comparison for demonstrating the effect of retrieved operational experience.
- Retrieved operational memory section for inspecting recalled evidence.
- Memory evidence page showing the configured memory backend and resolved experiences.
- Status indicators for Hindsight connectivity and LLM configuration.

11. Example Incident Flow

The reference demonstration uses a Payment API incident titled “Database connection timeout.” The displayed evidence includes a connection-pool timeout, latency spikes, exhausted database connections, increased 5xx responses, and a recent connection-pool configuration change. The engineer can request a memory-first investigation, inspect the

retrieved operational evidence, review the recommendation, and then approve the resolution.

The interface is designed so that the recommendation is inspectable rather than opaque. The before-versus-after comparison also allows the same incident to be analyzed with and without retrieved memory, making the contribution of historical context visible.

12. Fallback and Reliability Design

DeployMind does not require every incident to have a historical precedent. When no sufficiently relevant memory is found, the agent uses current incident evidence and deterministic investigation plans. The supplied AgentService contains evidence-first plans for incident categories including database, authentication, Redis/cache, messaging, memory, CPU, storage, network, HTTP server errors, and latency.

This design prevents the system from manufacturing a historical explanation simply to complete a demonstration. It also makes the system easier to test because the same incident can be evaluated with memory enabled or disabled.

13. Security and Authority Boundary

- The LLM is advisory and does not execute production actions.
- A human engineer is required to approve the production resolution.
- Current incident facts are treated as the primary source of truth.
- Historical memory is explicitly treated as supporting evidence.
- The model is instructed not to invent infrastructure state, metrics, logs, capacity, or completed actions.
- External provider configuration is kept outside the application logic through environment variables.

14. Running the Project

The supplied project uses Java 17 for the backend and Node.js for the React frontend. The backend can be started from IntelliJ by opening the backend folder as a Maven project, selecting JDK 17, reloading Maven, and running `DeployMindApplication`. The backend is configured to start on port 8000.

For the real LLM path, the backend environment should contain `GROQ_API_KEY` and, when required, `LLM_MODEL`. For real Hindsight memory, `HINDSIGHT_API_KEY` and `HINDSIGHT_BANK_ID` should be configured. If Hindsight is not configured, the local demo memory implementation is used.

The frontend is started from the frontend directory with `npm install` followed by `npm run dev`. The Vite frontend uses `VITE_API_URL` when supplied; otherwise, it uses the backend address configured in the project.

15. Current Limitations

- The supplied backend stores incidents in an in-memory collection rather than a production database.
- The local memory implementation is intended as a development/demo fallback and is not a replacement for the Hindsight path.
- The current project is a demo-oriented incident-response system rather than a complete production operations platform.
- The final production remediation is intentionally human-controlled rather than automatically executed.

16. Conclusion

DeployMind provides a clear architecture for combining incident evidence, persistent operational memory, and structured AI reasoning while retaining human authority over production changes. Its central design is the explicit incident-to-memory loop: current evidence is investigated with relevant historical experience, the AI produces an inspectable recommendation, a human approves the outcome, and the verified resolution becomes reusable operational memory.

The supplied architecture reference emphasizes that external capabilities remain behind application-level interfaces. Hindsight provides memory, Groq provides reasoning, and Spring Boot controls the incident lifecycle. This separation provides a practical foundation for extending the system without changing the central idea that resolved incidents should become reusable operational experience.

References

- DeployMind source project supplied as DeployMind-LLM-Hindsight-Final(1).zip.
- DeployMind Architecture Article, supplied as DeployMind Architecture Article.pdf.
- Project source files: Spring Boot controller, AgentService, HindsightMemoryService, LlmService, Incident model, React frontend, and configuration files.